

第 1 章

软件模式辨析

1 创建型模式

1.1 简单工厂

1.2 工厂方法

1.3 抽象工厂

2 结构型模式

2.1 适配器

将一个类的接口转换成客户期望的另一个接口。翻译官。

```
#include <iostream>
#include <string>

// 目标接口（客户端期望的）
class ILogger {
public:
    virtual ~ILogger() = default;
    virtual void log(const std::string& level, const std::string& message) = 0;
};

// 适配者（需要被适配的现有类）
class OldLogger {
public:
    void writeLog(const std::string& msg) {
        std::cout << "[OLD]" << msg << std::endl;
    }
};

// 适配器类：实现目标接口，内部调用适配者
class LoggerAdapter : public ILogger {
    OldLogger* oldLogger; // 持有适配者
public:
    LoggerAdapter(OldLogger* logger) : oldLogger(logger) {}

    void log(const std::string& level, const std::string& message) override {
        // 转换：把 level 和 message 拼成旧接口能接受的格式
        std::string formattedMsg = "[" + level + "]" + message;
        oldLogger->writeLog(formattedMsg);
    }
};

// 客户端代码（只依赖 ILogger）
```

```

void process(ILogger* logger) {
    logger->log("INFO", "用户登录");
    logger->log("ERROR", "数据库连接失败");
}

int main() {
    OldLogger old; // 老库的对象
    LoggerAdapter adapter(&old); // 用适配器包装

    process(&adapter); // 客户端可以用 ILogger 接口了

    return 0;
}

```

客户端通过 ILogger 接口调用，实际上执行的是 OldLogger 的功能。适配器完成了接口转换。

2.2 装饰器

```

#include <iostream>
#include <memory>
#include <string>

// 1. 抽象组件：饮料
class Beverage {
public:
    virtual ~Beverage() = default;
    virtual std::string getDescription() const = 0;
    virtual double cost() const = 0;
};

// 2. 具体组件：浓缩咖啡
class Espresso : public Beverage {
public:
    std::string getDescription() const override {
        return "浓缩咖啡";
    }
    double cost() const override {
        return 1.99;
    }
};

// 3. 抽象装饰器
class CondimentDecorator : public Beverage {
protected:
    std::unique_ptr<Beverage> beverage; // 持有被装饰的对象
public:
    CondimentDecorator(std::unique_ptr<Beverage> bev) :
    beverage(std::move(bev)) {}
};

// 4. 具体装饰器A：牛奶
class Milk : public CondimentDecorator {
public:
    Milk(std::unique_ptr<Beverage> bev) : CondimentDecorator(std::move(bev)) {}
    std::string getDescription() const override {
        return beverage->getDescription() + "☐+☐牛奶";
    }
};

```

```

    }
    double cost() const override {
        return beverage->cost() + 0.30;
    }
};

// 具体装饰器B: 摩卡 (巧克力味)
class Mocha : public CondimentDecorator {
public:
    Mocha(std::unique_ptr<Beverage> bev) : CondimentDecorator(std::move(bev))
    {}

    std::string getDescription() const override {
        return beverage->getDescription() + "└┐摩卡";
    }

    double cost() const override {
        return beverage->cost() + 0.50;
    }
};

// 客户端使用
int main() {
    // 点一杯浓缩咖啡
    std::unique_ptr<Beverage> drink = std::make_unique<Espresso>();
    std::cout << drink->getDescription() << "└┐价格:└┐$" << drink->cost() <<
std::endl;

    // 加双份牛奶 (先加一层牛奶, 再加一层牛奶)
    drink = std::make_unique<Milk>(std::move(drink));
    drink = std::make_unique<Milk>(std::move(drink));
    std::cout << drink->getDescription() << "└┐价格:└┐$" << drink->cost() <<
std::endl;

    // 再叠加摩卡
    drink = std::make_unique<Mocha>(std::move(drink));
    std::cout << drink->getDescription() << "└┐价格:└┐$" << drink->cost() <<
std::endl;

    return 0;
}

```

Q: 那我为什么不在原味奶茶这个类里再写两个布尔类型的成员变量, 一个是是否有珍珠, 一个是是否有椰果?

A: 违反开闭原则, 类会无限膨胀!

2.3 外观

为复杂子系统提供一个统一的、简化的接口, 让客户端更容易使用该子系统。

比如电脑开机按钮: 你按一下电源键, 背后要启动 CPU、内存、硬盘、显卡.....但你只需要按一个键。

2.4 代理

为另一个对象提供一个替身或占位符, 以控制对这个对象的访问。代理对象与原对象实现相同的接口, 并在内部持有原对象的引用 (或在需要时才创建原对象)。

简单说：不直接操作目标对象，而是通过一个**中间人**（代理）来操作。这个中间人可以在操作前后做额外的事情（权限检查、延迟加载、日志记录等）。

比如：律师代理、房产中介

三种常见代理类型（用对比表格先建立直觉）

代理类型	主要控制目的	生活类比
虚拟代理	延迟创建大对象（节省资源）	网页上的图片占位符，滚动到可见时才真正加载
保护代理	检查访问权限	公司门禁卡：只有特定员工才能进入某些区域
远程代理	隐藏对象在另一台服务器上	外卖平台：你在App下单，代理帮你联系店家

3 行为型模式

3.1 策略

把“做什么”和“怎么做”分开。客户端只需要决定“做什么”（比如计算价格、排序、压缩），而具体的“怎么做”（算法）可以随时更换，互不影响。

比如：去A到B的不同交通策略，不同支付方式的支付策略

3.2 模板方法

在一个父类中定义一个算法的骨架（模板），将一些步骤延迟到子类中实现。子类可以在不改变算法整体结构的前提下，重定义算法的某些特定步骤。

简单说：把“不变的部分”写在父类里，“变化的部分”交给子类去填空。

比如：做咖啡和茶：步骤都是 ①烧水 ②冲泡（咖啡/茶叶） ③倒杯 ④加调料。做咖啡和茶的骨架一样，只是第二步和

考试答题：所有考生都按同一流程——发卷、读题、答题、交卷。但每个人答题的内容不同。父类固定流程，子类实现具体答题。

3.3 观察者

定义对象间的一对多依赖关系，当一个对象（主题）的状态发生变化时，所有依赖它的对象（观察者）都会自动收到通知并更新。

简单说：“发布-订阅”。一个对象（发布者）状态变了，所有关注它的对象（订阅者）都能自动得到通知，无需发布者知道订阅者是谁。

比如：微信公众号。

```
#include <iostream>
#include <vector>
#include <algorithm>

// 观察者接口
class Observer {
```

```

public:
    virtual ~Observer() = default;
    virtual void update(float temperature, float humidity) = 0;
};

// 主题接口
class Subject {
public:
    virtual ~Subject() = default;
    virtual void registerObserver(Observer* o) = 0;
    virtual void removeObserver(Observer* o) = 0;
    virtual void notifyObservers() = 0;
};

// 具体主题：气象站
class WeatherStation : public Subject {
    std::vector<Observer*> observers;
    float temperature;
    float humidity;
public:
    void registerObserver(Observer* o) override {
        observers.push_back(o);
    }
    void removeObserver(Observer* o) override {
        auto it = std::find(observers.begin(), observers.end(), o);
        if (it != observers.end()) observers.erase(it);
    }
    void notifyObservers() override {
        for (auto* observer : observers) {
            observer->update(temperature, humidity);
        }
    }
    // 气象站数据变化时调用
    void setMeasurements(float temp, float hum) {
        temperature = temp;
        humidity = hum;
        measurementsChanged();
    }
    void measurementsChanged() {
        notifyObservers();
    }
};

// 具体观察者A：当前天气显示屏
class CurrentDisplay : public Observer {
public:
    void update(float temperature, float humidity) override {
        std::cout << "当前天气: 温度" << temperature
            << "°C, 湿度" << humidity << "%\n";
    }
};

// 具体观察者B：天气预报显示屏（简化：根据上次值预测）
class ForecastDisplay : public Observer {
    float lastTemp = 0;
public:

```

```

        void update(float temperature, float humidity) override {
            std::string forecast = (temperature > lastTemp) ? "升温" : "降温";
            lastTemp = temperature;
            std::cout << "天气预报:_" << forecast << std::endl;
        }
};

// 具体观察者C: 统计显示屏 (简化: 只记录最高温)
class StatisticsDisplay : public Observer {
    float maxTemp = -100;
public:
    void update(float temperature, float humidity) override {
        if (temperature > maxTemp) maxTemp = temperature;
        std::cout << "统计:_最高温度_" << maxTemp << "°C\n";
    }
};

// 客户端
int main() {
    WeatherStation station;
    CurrentDisplay current;
    ForecastDisplay forecast;
    StatisticsDisplay stats;

    station.registerObserver(&current);
    station.registerObserver(&forecast);
    station.registerObserver(&stats);

    std::cout << "第一组数据:\n";
    station.setMeasurements(25.0, 65.0);
    std::cout << "\n第二组数据:\n";
    station.setMeasurements(28.0, 70.0);

    // 移除一个观察者
    station.removeObserver(&forecast);
    std::cout << "\n移除天气预报屏后, 第三组数据:\n";
    station.setMeasurements(22.0, 60.0);

    return 0;
}

```

3.4 访问者

表示一个作用于某对象结构中的各元素的操作。它让你可以在不改变各元素类的前提下，定义作用于这些元素的新操作。

简单说：**把“数据”和“对数据的操作”分离**。当你想给一组对象添加新功能时，不修改这些对象的类，而是写一个独立的“访问者”类，让每个对象“接待”这个访问者，由访问者完成操作。

例子：体检中心，你（元素）去体检，不同的医生（访问者：眼科、内科、牙科）给你做不同检查。你的身体结构不变，但可以接待不同医生。

```

#include <iostream>
#include <vector>

```

```

// 前向声明
class File;
class Directory;

// 访问者接口
class Visitor {
public:
    virtual ~Visitor() = default;
    virtual void visit(File* file) = 0;
    virtual void visit(Directory* dir) = 0;
};

// 元素接口
class Element {
public:
    virtual ~Element() = default;
    virtual void accept(Visitor* visitor) = 0;
};

// 具体元素: 文件
class File : public Element {
    std::string name;
    int size;
public:
    File(const std::string& n, int s) : name(n), size(s) {}
    int getSize() const { return size; }
    std::string getName() const { return name; }
    void accept(Visitor* visitor) override {
        visitor->visit(this);    // 双分派: 将自己传递给访问者
    }
};

// 具体元素: 文件夹
class Directory : public Element {
    std::string name;
    std::vector<Element*> children;
public:
    Directory(const std::string& n) : name(n) {}
    void add(Element* e) { children.push_back(e); }
    const std::vector<Element*>& getChildren() const { return children; }
    std::string getName() const { return name; }
    void accept(Visitor* visitor) override {
        visitor->visit(this);    // 先处理自己
        for (auto* child : children) {
            child->accept(visitor); // 递归让子元素接受访问者
        }
    }
};

// 具体访问者1: 计算总大小
class SizeCalculator : public Visitor {
    int totalSize = 0;
public:
    void visit(File* file) override {
        totalSize += file->getSize();
    }
};

```

```

    void visit(Directory* dir) override {
        // 文件夹本身大小不计算，只遍历子元素（已在 accept 中处理）
        // 这里可以留空或打印日志
    }
    int getTotalSize() const { return totalSize; }
};

// 具体访问者2：病毒扫描（输出感染警告）
class VirusScanner : public Visitor {
public:
    void visit(File* file) override {
        // 模拟扫描：根据文件名判断
        if (file->getName().find("virus") != std::string::npos) {
            std::cout << "发现病毒文件:_" << file->getName() << "\n";
        }
    }
    void visit(Directory* dir) override {
        // 检查文件夹名是否可疑
        if (dir->getName() == "System32") {
            std::cout << "警告：可疑系统文件夹_" << dir->getName() << "\n";
        }
    }
};

// 具体访问者3：生成报告（列出所有文件路径）
class ReportGenerator : public Visitor {
    int depth = 0;
public:
    void visit(File* file) override {
        for (int i = 0; i < depth; ++i) std::cout << "_";
        std::cout << "文件:_" << file->getName() << "\n";
    }
    void visit(Directory* dir) override {
        for (int i = 0; i < depth; ++i) std::cout << "_";
        std::cout << "目录:_" << dir->getName() << "\n";
        depth++;
        // 注意：子元素的访问在 Directory::accept 中递归进行，这里无需再遍历
        // 为了正确缩进，需要在 accept 递归前增加深度，递归后减少深度。
        // 更规范的做法是在 ReportGenerator 中维护一个栈，这里简化处理。
    }
};

// 修正 ReportGenerator 的深度控制（为正确性，重写如下：）
class ReportGeneratorFixed : public Visitor {
    int depth = 0;
public:
    void visit(File* file) override {
        for (int i = 0; i < depth; ++i) std::cout << "_";
        std::cout << "文件:_" << file->getName() << "\n";
    }
    void visit(Directory* dir) override {
        for (int i = 0; i < depth; ++i) std::cout << "_";
        std::cout << "目录:_" << dir->getName() << "\n";
        depth++;
        // 注意：此处不负责递归，递归由 Directory::accept 完成。
        // 但 accept 调用时会再次调用 visit(Directory)，导致无限递归。
    }
};

```



```

        // 实际上标准做法是: Directory::accept 中先调用 visitor->visit(this),
        // 然后再遍历子元素并调用 child->accept(visitor)。在 visit(Directory) 中
        // 不要再次遍历子元素。
        // 所以上面的写法是正确的, 但深度需要在递归结束后减回去。然而, 因为 accept
        // 会递归调用, 所以 depth 的变化需要在不同调用间传递。
        // 更清晰的方式: 在 Directory::accept 中处理深度增减, 或者使用栈。为简化
        // 我们不在本例中实现完美缩进。
    }
};
// 为了避免复杂, 我们直接使用前两个访问者演示核心概念。

// 客户端
int main() {
    File* f1 = new File("readme.txt", 100);
    File* f2 = new File("virus.exe", 500);
    Directory* root = new Directory("root");
    Directory* sub = new Directory("subdir");
    File* f3 = new File("data.doc", 200);

    sub->add(f3);
    root->add(f1);
    root->add(f2);
    root->add(sub);

    // 计算总大小
    SizeCalculator sizeCalc;
    root->accept(&sizeCalc);
    std::cout << "总大小: " << sizeCalc.getTotalSize() << " bytes\n";

    // 病毒扫描
    VirusScanner scanner;
    root->accept(&scanner);

    // 清理内存 (略)
    return 0;
}

```

3.5 状态

允许一个对象在其内部状态改变时改变它的行为。把“状态”和“行为”绑定。一个对象有多个状态, 不同状态下对同一请求的响应不同。状态模式将每个状态的行为封装到独立的类中, 让对象的行为随着内部状态的改变而自动切换。

比如, 电梯: 电梯有开门、关门、运行、停止等状态。按“关门”按钮: 如果在开门状态, 电梯关门; 如果在运行状态, 按钮无效。不同状态下, 同一按钮的行为不同。

```

#include <iostream>
#include <memory>

// 前向声明 Context
class Door;

// 状态接口
class DoorState {
public:
    virtual ~DoorState() = default;

```

```

        virtual void open(Door* door) = 0;
        virtual void close(Door* door) = 0;
        virtual void lock(Door* door) = 0;
        virtual void unlock(Door* door) = 0;
};

// 上下文: 门
class Door {
    std::unique_ptr<DoorState> state;
public:
    Door(std::unique_ptr<DoorState> initial_state);
    void setState(std::unique_ptr<DoorState> new_state);
    void open() { state->open(this); }
    void close() { state->close(this); }
    void lock() { state->lock(this); }
    void unlock() { state->unlock(this); }
};

// 具体状态: 打开状态
class OpenState : public DoorState {
public:
    void open(Door* door) override {
        std::cout << "门已经是打开的\n";
    }
    void close(Door* door) override {
        std::cout << "关闭门\n";
        door->setState(std::make_unique<ClosedState>());
    }
    void lock(Door* door) override {
        std::cout << "门开着, 无法上锁, 请先关门\n";
    }
    void unlock(Door* door) override {
        std::cout << "门开着, 无需解锁\n";
    }
};

// 具体状态: 关闭状态
class ClosedState : public DoorState {
public:
    void open(Door* door) override {
        std::cout << "打开门\n";
        door->setState(std::make_unique<OpenState>());
    }
    void close(Door* door) override {
        std::cout << "门已经是关闭的\n";
    }
    void lock(Door* door) override {
        std::cout << "门上锁\n";
        door->setState(std::make_unique<LockedState>());
    }
    void unlock(Door* door) override {
        std::cout << "门未上锁, 无需解锁\n";
    }
};

// 具体状态: 上锁状态

```

```

class LockedState : public DoorState {
public:
    void open(Door* door) override {
        std::cout << "门锁着, 无法打开\n";
    }
    void close(Door* door) override {
        std::cout << "门已经关闭且锁着\n";
    }
    void lock(Door* door) override {
        std::cout << "门已经是锁着的\n";
    }
    void unlock(Door* door) override {
        std::cout << "解锁门\n";
        door->setState(std::make_unique<ClosedState>());
    }
};

// 实现 Door 构造函数和方法
Door::Door(std::unique_ptr<DoorState> initial_state) : state(std::move(initial_state)) {}
void Door::setState(std::unique_ptr<DoorState> new_state) {
    state = std::move(new_state);
}

// 客户端
int main() {
    Door door(std::make_unique<ClosedState>());

    door.open();    // 打开
    door.lock();    // 开着时无法上锁
    door.close();   // 关闭
    door.lock();    // 上锁
    door.open();    // 锁着无法打开
    door.unlock();  // 解锁
    door.open();    // 现在可以打开了

    return 0;
}

```